

Post-training & Reinforcement Learning II

DPO and GRPO

Aaron Mueller

CS599 B1: Advanced Natural Language Processing

Oct. 1, 2026

Administrivia

- The final project details are live! See the course website and Piazza.
 - First checkpoint: the project proposal, due **Oct. 20**
- Reminder: HW2 is due in 2 weeks, on **Oct. 15**

Recap: PPO

- PPO requires us to keep track of four independent sets of language model parameters:
 - A policy π_θ : the LM we're post-training
 - A reference policy π_{ref} : the original LM
 - A value function V_ϕ : an LM that estimates baseline rewards
 - A reward model r_ψ : an LM that computes scalar rewards

Direct Preference Optimization (DPO)

Motivation

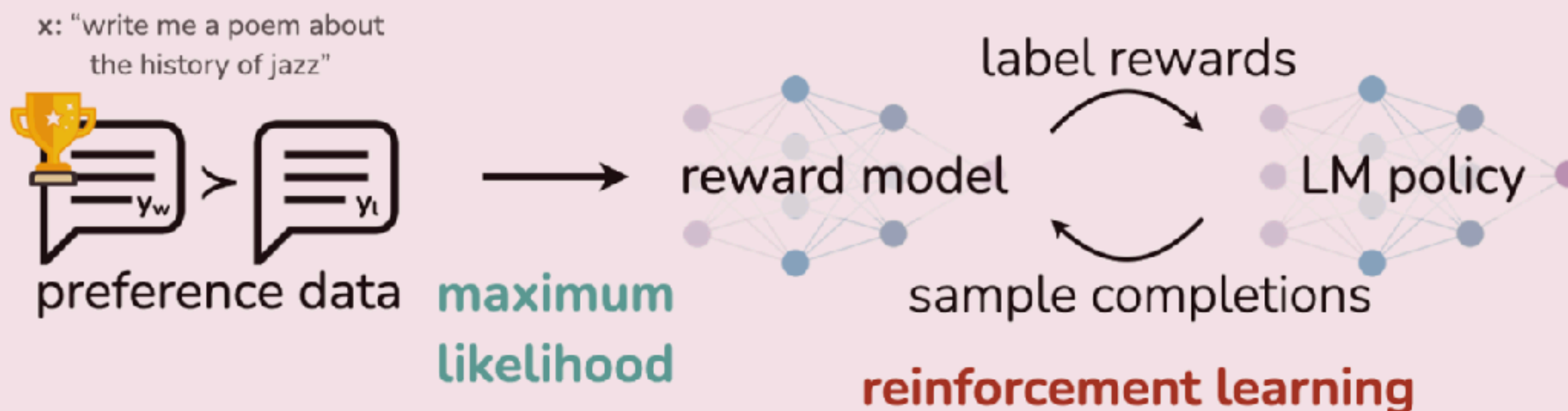
- You will be happy to learn that **direct preference optimization (DPO)** is way simpler
 - Greatly increased the accessibility of RLHF and post-training research in general
- DPO differs from PPO in the following ways:
 - No separate reward model
 - No reinforcement learning loop
 - Applies one loss directly to the LM given preference pairs

DPO vs. PPO

- RLHF involves:
 - Collecting human preference data
 - Training a reward model $r_\phi(x, y)$
 - Optimizing a policy against r_ϕ with RL
- DPO collapses steps 2 and 3 into **one supervised loss**

[Rafailov et al., 2024]

Reinforcement Learning from Human Feedback (RLHF)



Direct Preference Optimization (DPO)



Deriving DPO

- We start with a similar reinforcement learning objective to PPO:

$$\arg \max_{\theta} \mathbb{E}_{x \sim D, y \sim \pi_{\theta}(y|x)} \left[\underbrace{r(x, y)}_{\text{Maximize reward}} - \underbrace{\beta \text{KL}[\pi_{\theta}(y|x), \pi_{\text{ref}}(y|x)]}_{\text{Don't stray too far from the original model}} \right]$$

- Theoretically speaking, the optimal policy π^* is: $\pi^* = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp(\frac{1}{\beta} r(x, y))$
- We can rearrange this to get: $r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$

Deriving DPO

$$r(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x)$$

- Now let's combine this with our Bradley-Terry model from last lecture:

$$p^*(y_w \succ y_l | x) = \frac{1}{1 + \exp\left(\beta \log \frac{\pi^*(y_l | x)}{\pi_{\text{ref}}(y_l | x)} - \beta \log \frac{\pi^*(y_w | x)}{\pi_{\text{ref}}(y_w | x)}\right)}$$

- This equation says the preferred output y_w should be more likely under our learned policy than under the reference, and dispreferred output y_l should be less likely
- Now we can learn the policy to optimize the log-likelihood of the preference data:

$$L_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = - \mathbb{E}_{(x, y_w, y_l) \sim D} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

Deriving DPO


$$L_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = - \mathbb{E}_{(x, y_w, y_l) \sim D} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

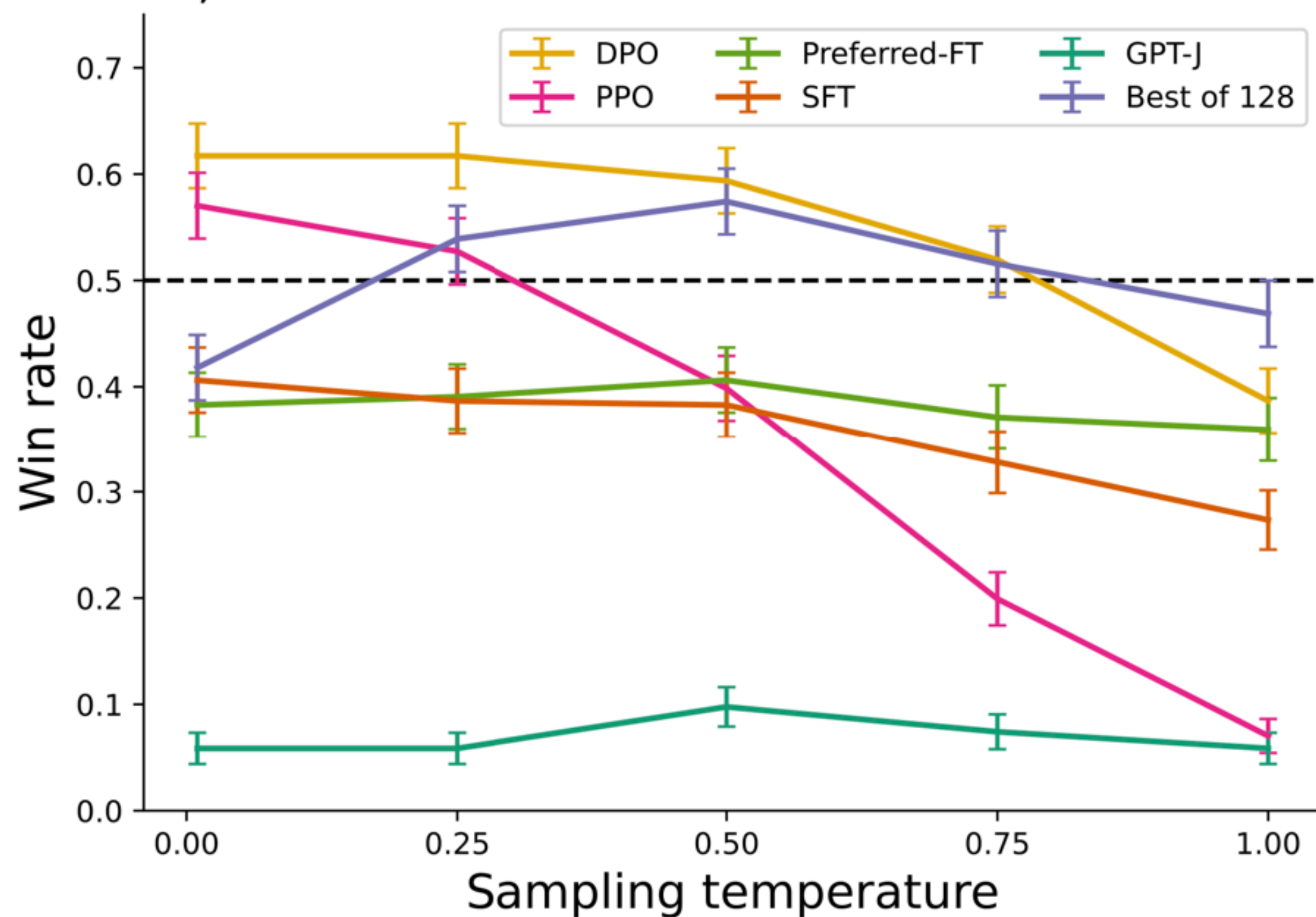

- Or perhaps more intuitively:

$$L_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = - \mathbb{E}_{(x, y_w, y_l) \sim D} \left[\log p(y_w \succ y_l | x) \right]$$

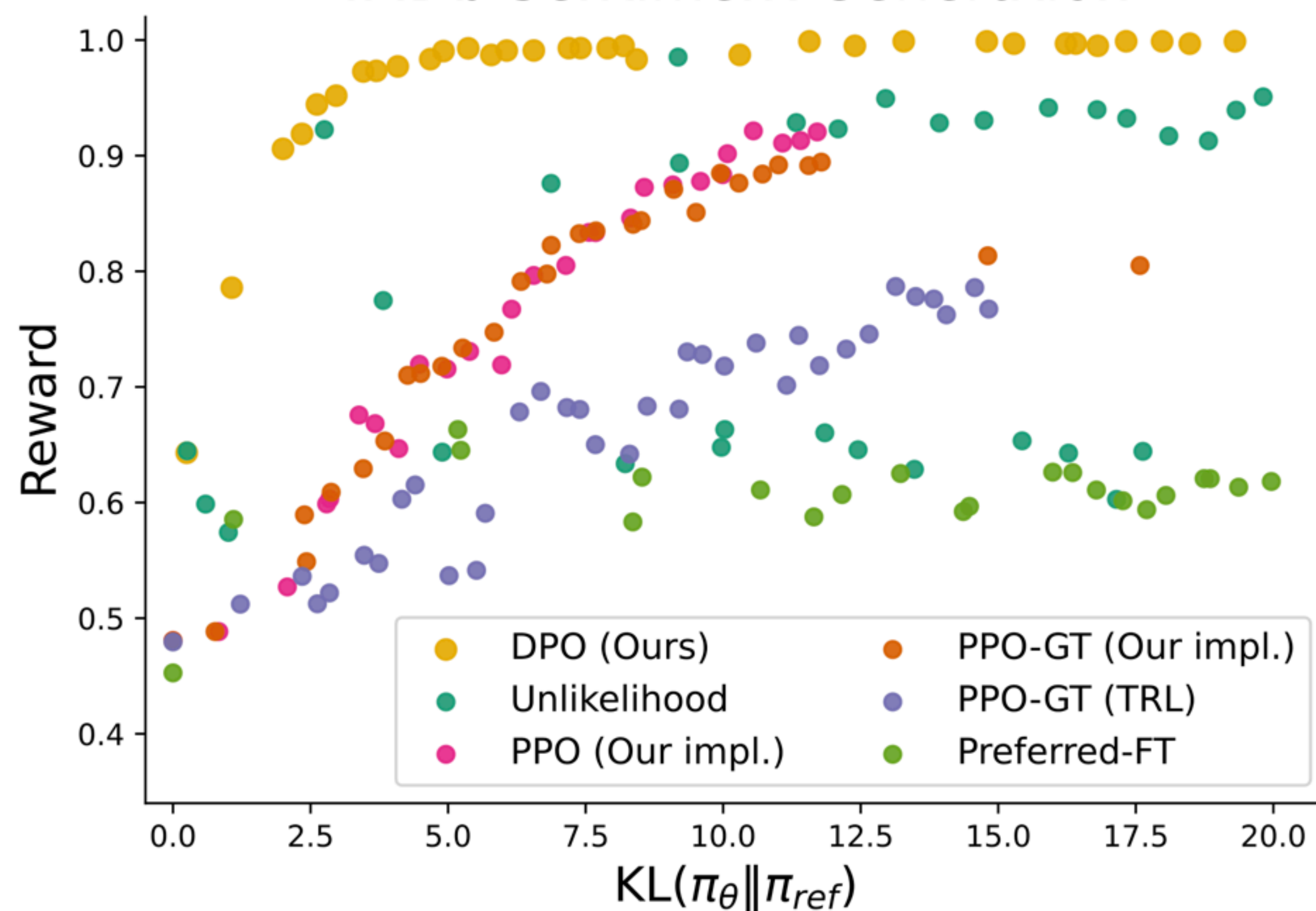
- Given preference pairs, this is differentiable as-is! *No reward model, sampling, or reinforcement learning loop necessary*
- Also notice: we only need to load *two* language models in memory now: the policy π_{θ} and the reference model π_{ref}

DPO vs. PPO

TL;DR Summarization Win Rate vs Reference



IMDb Sentiment Generation



Why does DPO work?

- Key insight: The optimal RLHF policy and the reward model are actually the same thing! If we know one, we know the other
 - This means we can train the policy *directly* on preferences
- As we saw when deriving the DPO loss given the original objective, terms cancel very nicely given the above fact

Online vs. Offline Learning

- PPO and REINFORCE are **online**: generates fresh completions during training, scored with a reward model
 - Can explore new regions, meaning higher peak performance is possible, but requires many more moving parts and more memory
- DPO is **offline**: train on a fixed dataset collected ahead of time
 - Simpler, more stable, but limited by the scope of the training data

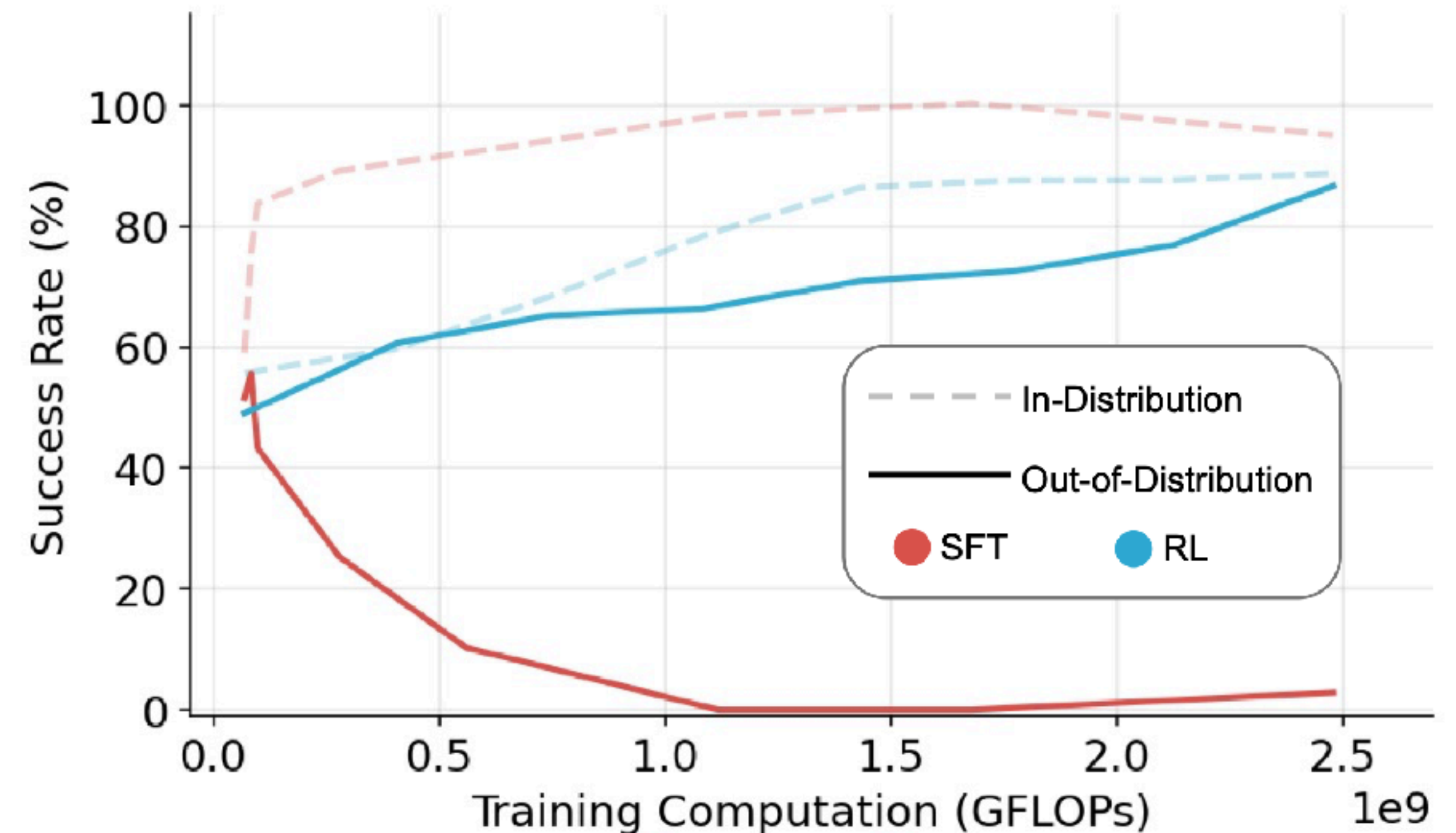
DPO vs. SFT

- SFT involves predicting the correct next token, or a correct label
 - Learning via mimicry
 - Doesn't show model directly what a *bad* response looks like
- DPO involves optimizing pairwise preferences
 - Learning via contrast
 - Optimize preferences while not straying too far from original model

“SFT memorizes, RL generalizes”

- It's commonly believed that SFT leads to worse generalizes because it is more prone to memorizing the training data, whereas RL is less prone to this
- In many tasks, RL improves out-of-distribution accuracy, while SFT causes a massive reduction in out-of-distribution accuracy if you train long enough

[Chu et al., 2025]



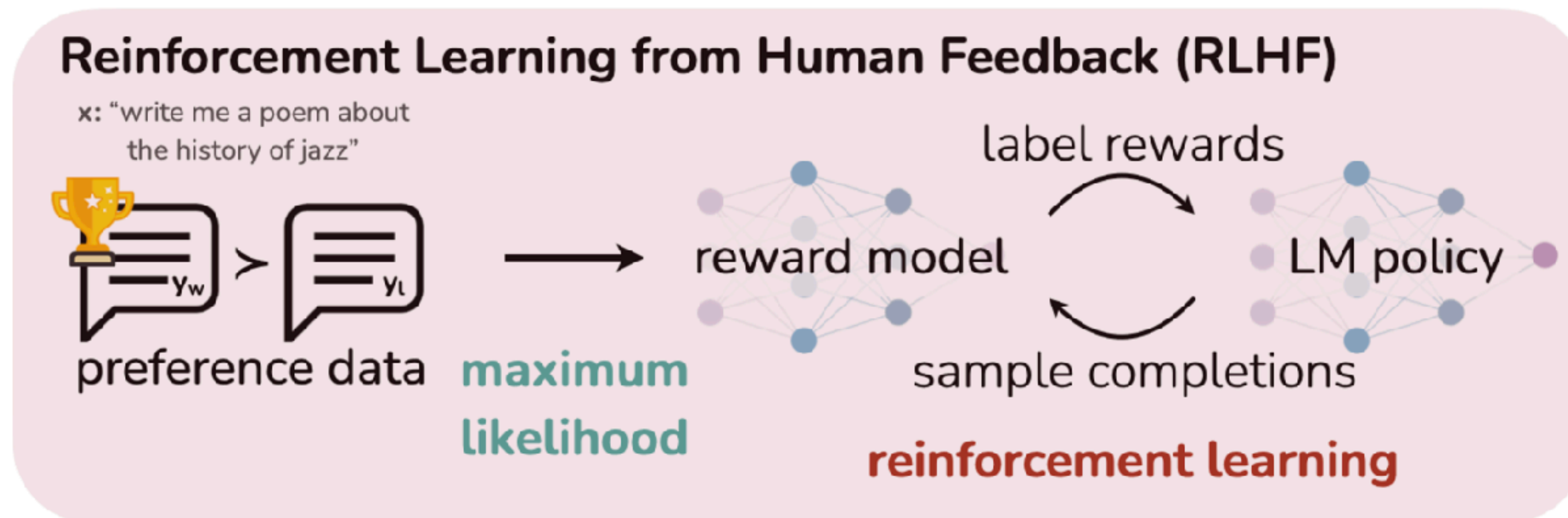
Why is DPO rare today?

- DPO works well when we're doing distillation
- DPO doesn't perform quite as well as PPO in practice
 - If a frontier lab has the resources, they'll probably do something that performs better empirically
- Basically, DPO gets us a *good* model, but not the *best possible* model
- However, some models like Olmo 3 still sneak DPO in as an intermediate step during post-training

GRPO and RLVR: Toward Reasoning Models

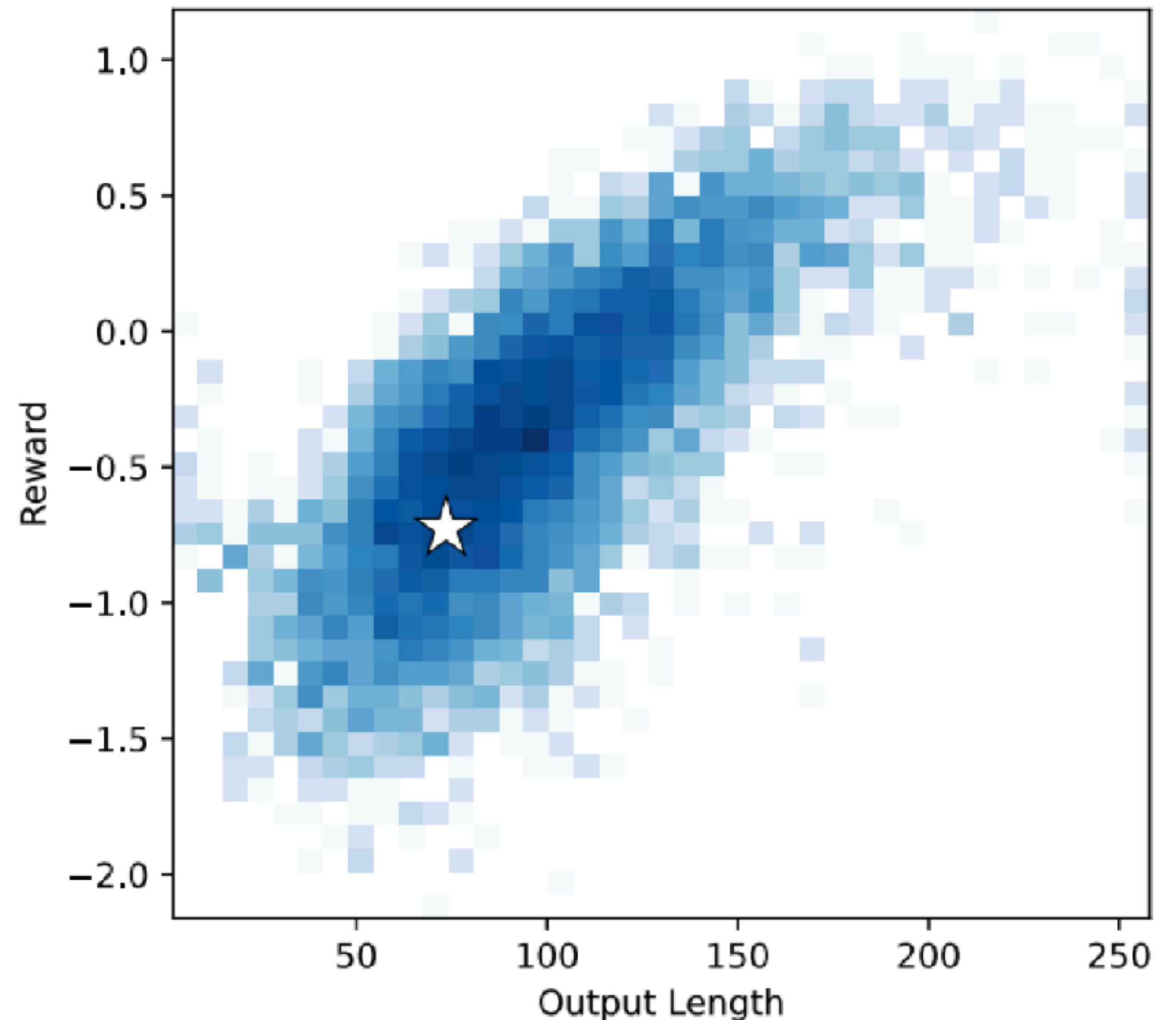
RLHF

- So far, we've focused on RLHF: *subjective* pairwise preference scoring
- Allows us to optimize toward some goal, even if that goal is underdefined



RLHF Downsides

- Scalability: could you imagine doing a pre-training-scale RLHF run?
- It is easy to misspecify rewards when using pairwise preferences
 - Think about what spurious correlations might be hiding in human preferences...



Reinforcement Learning with Verifiable Rewards (RLVR)

- These days, many models instead use **reinforcement learning with verifiable rewards (RLVR)**
 - Verifiable reward: problem has a *verifiable* answer, and we can say definitively whether the answer was correct or not
 - No need for a neural/LLM reward model; we can compute rewards deterministically
- In what settings will we have verifiable solutions?
 - Math
 - Coding
 - Agentic tasks

RLHF vs. RLVR

RLHF

RLVR

Task Settings

“Chat” problems

Math, coding, agentic tasks

Training Data

Prompt x , preferred response y_w , dispreferred response y_l

Prompt x , answer y

Rewards

Language model-based

Verifiable: 1 or 0, correct or incorrect

Algorithms

PPO, DPO

PPO, GRPO

RLVR Timeline

When and why did RLVR become mainstream compared to RLHF?

Language tasks: translation,
QA, summarization

2020

“Chat” tasks:
open-ended QA

2022

Verifiable tasks: math,
coding, agentic tasks

2024

To a large extent, this reflects changes in where and how LLMs are being deployed

Policy Gradient

- Our objective is: $\sum_{\tau} r(\tau)p(\tau | \theta)$
- We sample trajectories τ : run the LM to get output $\hat{y}$ given input x
 - Reminder: we view the output as a series of actions a (generated tokens) given environment states s (previous tokens)

- Compute the return $r(\tau)$

- Compute the gradient:

$$\mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \left(\underbrace{r(\tau)}_{\text{Reward}} - \underbrace{b(s_t)}_{\text{Baseline}} \right) \right]$$

$\approx$ SFT loss on the generated sequence

Group Relative Policy Optimization (GRPO)

- Recall our PPO objective:

Sample problems $\mathbf{q}$, then
sample G rollouts $\mathbf{o}_i$ from
the policy.

$$\rho_t = \frac{\pi_{\theta}(\mathbf{o}_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(\mathbf{o}_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})}$$

$$J(\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \cdot \hat{A}_t \right) \right]$$

- GRPO has the same general form, but we change how many elements are computed: the ratio ρ , rewards r_t , advantages $\hat{A}_t$, and the KL

$$\hat{A}_{i,t} = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}$$

We normalize the reward relative to the group G .

GRPO

KL Penalty

$$\rho_{i,t} = \frac{\pi_{\theta}(\mathbf{o}_{i,t} \mid \mathbf{q}, \mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(\mathbf{o}_{i,t} \mid \mathbf{q}, \mathbf{o}_{i,<t})}$$

$$J(\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \cdot \hat{A}_t \right) - \beta \text{KL}(\pi_{\theta} \parallel \pi_{\text{ref}}) \right]$$

The modified KL divergence looks like this:

$$\text{KL}(\pi_{\theta} \parallel \pi_{\text{ref}}) = \frac{\pi_{\text{ref}}(\mathbf{o}_{i,t} \mid \mathbf{q}, \mathbf{o}_{i,<t})}{\pi_{\theta}(\mathbf{o}_{i,t} \mid \mathbf{q}, \mathbf{o}_{i,<t})} - \log \frac{\pi_{\text{ref}}(\mathbf{o}_{i,t} \mid \mathbf{q}, \mathbf{o}_{i,<t})}{\pi_{\theta}(\mathbf{o}_{i,t} \mid \mathbf{q}, \mathbf{o}_{i,<t})} - 1$$

This version has nicer properties (e.g., it's never negative).

(Also notice that it's folded into the expectation, rather than into the reward.)

GRPO vs. PPO

- Notice: the group mean in GRPO replaces the learned value function from PPO!

$$\hat{A}_{i,t} = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}$$

- This saves us a lot of compute and memory
- GRPO with RLVR thus requires loading *two* language models in memory:
 - The policy π_θ
 - The reference π_{ref}

Case Study: DeepSeek-R1-Zero

- Uses only reinforcement learning without instruction fine-tuning
- Uses GRPO and RLVR
- During training, use this template to wrap our input prompts:

Table 1 | Template for DeepSeek-R1-Zero. **prompt** will be replaced with the specific reasoning question during training.

A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. The reasoning process and answer are enclosed within `<think>...</think>` and `<answer>...</answer>` tags, respectively, i.e., `<think> reasoning process here </think>` `<answer> answer here </answer>`. User: **prompt**. Assistant:

Prompt format is a key ingredient of performance!

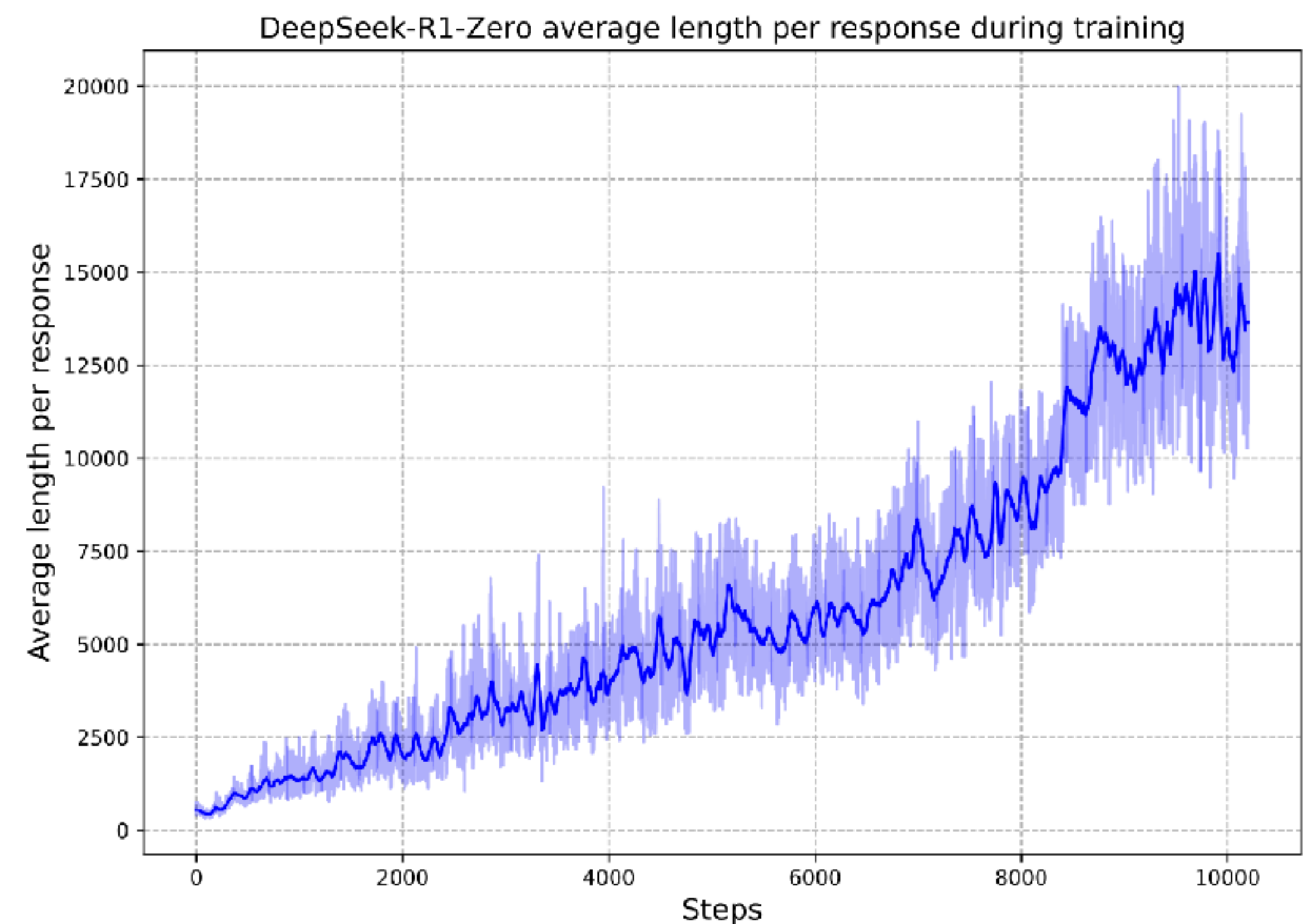
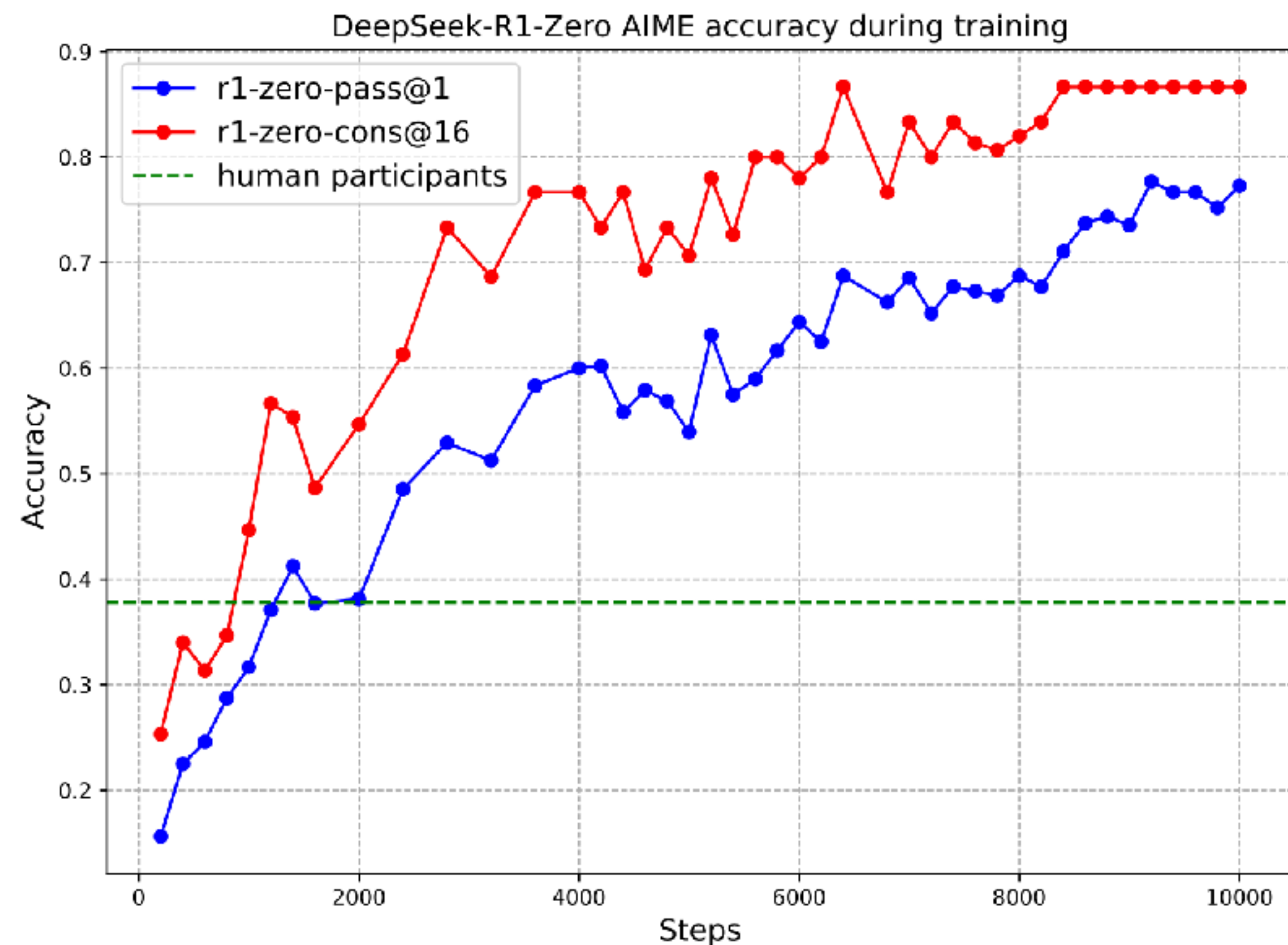
Case Study: DeepSeek-R1-Zero

- Two kinds of verifiable rewards:
 - **Accuracy rewards:** evaluates whether the response to the question is correct
 - **Format rewards:** enforces specific formatting requirements:
 - Reasoning process is contained within special `<think> ... </think>` tokens
- Combine these rewards with the same weight:

$$r = r_{\text{acc}} + r_{\text{format}}$$

Case Study: DeepSeek-R1-Zero

- The result of this is massively scalable inference and *long* chains of thought:



- RLVR enabled reasoning models and test-time scaling

Case Study: DeepSeek-R1-Zero

Table 2 | An interesting “aha moment” of an intermediate version of DeepSeek-R1-Zero. The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to

Response: <think>

To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both ...

$$\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2.$$

Rearrange to isolate the inner square root term:

$$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$$

...

Wait, wait. Wait. That's an aha moment I can flag here.

Let's reevaluate this step-by-step to identify if the correct sum can be ...

We started with the equation:

$$\sqrt{a - \sqrt{a + x}} = x$$

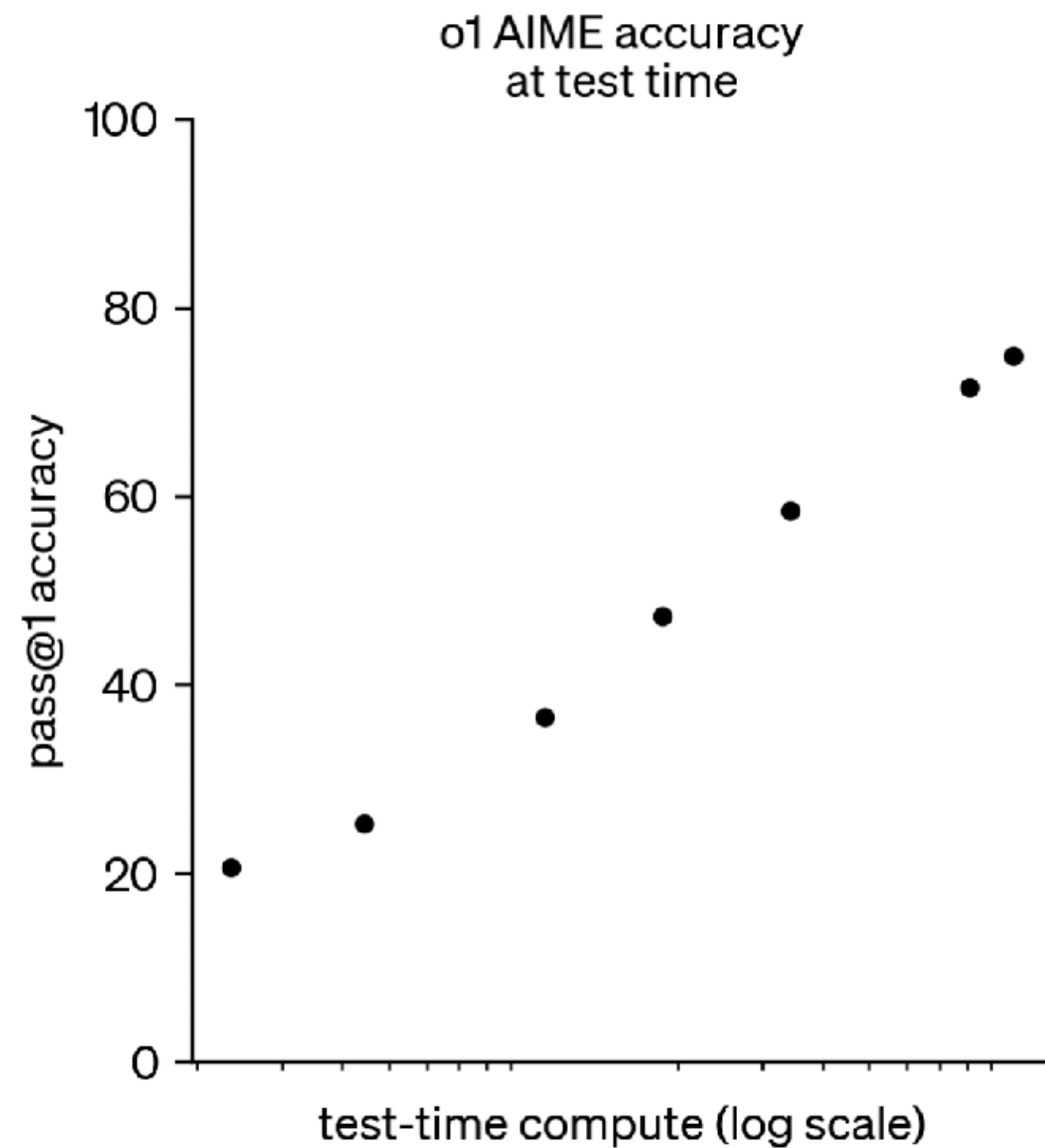
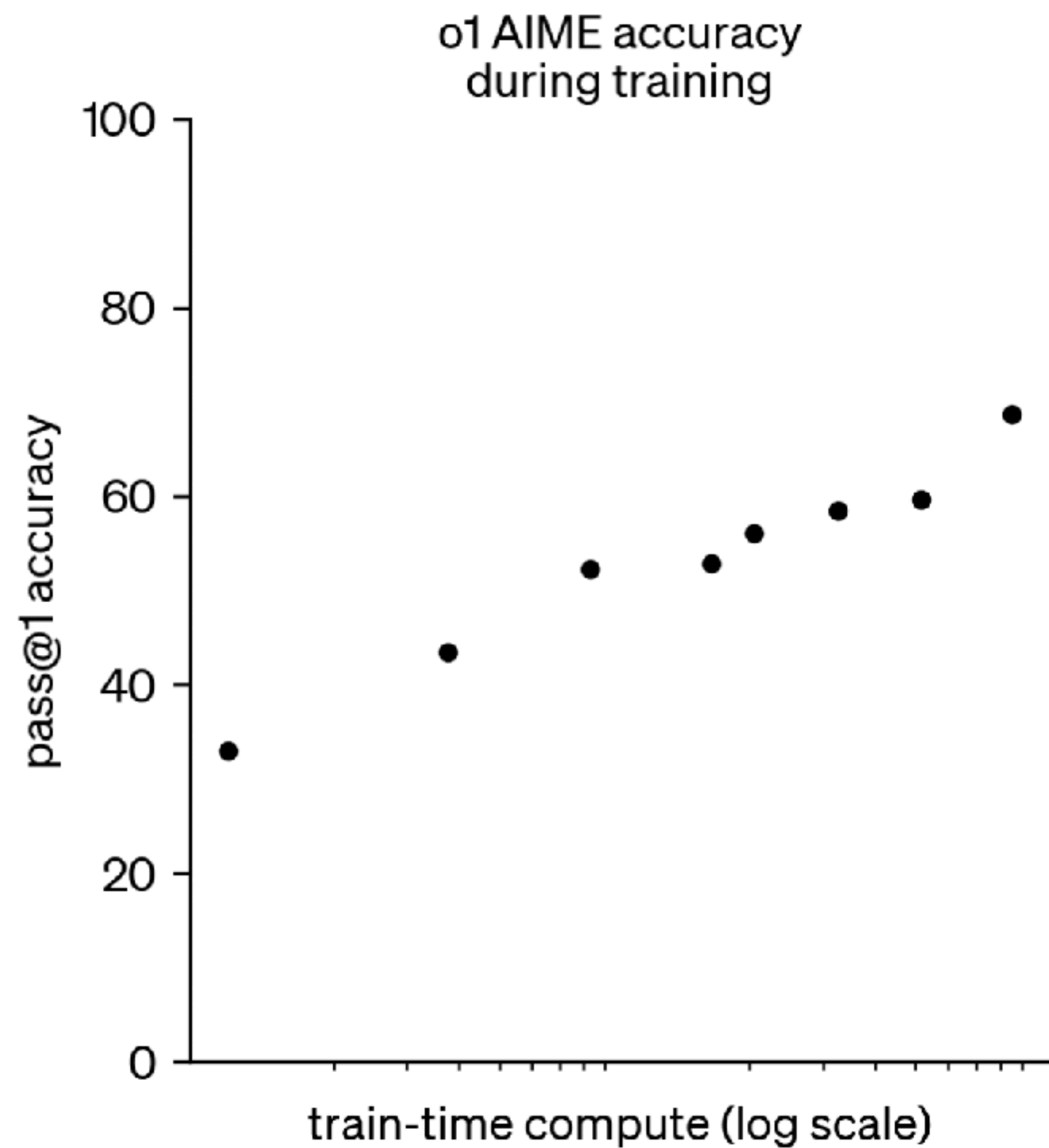
First, let's square both sides:

$$a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$$

Next, I could square both sides again, treating the equation: ...

...

Test-time Scaling



An Explosion of Reasoning Models



OpenAI o1
Sep 12, 2024



DeepSeek R1
Jan 20, 2025



Kimi 1.5
Jan 20, 2025



Open-Reasoner-Zero
Mar 31, 2025



Qwen 3
Apr 29, 2025



Phi-4 Reasoning
Apr 30, 2025



MiMo
Apr 30, 2025



Skywork OR-1
May 2025



INTELLECT-2
May 12, 2025



Open Thoughts
OpenThoughts 3
Jun 5, 2025



Magistral
Jun 10, 2025



MiniMax-M1
Jun 16, 2025



Kimi K2
Jul 11, 2025



GLM-4.5
Jul 28, 2025



OLMo 3 Think
Nov 20, 2025



DeepSeek V3.2
Dec 1, 2025



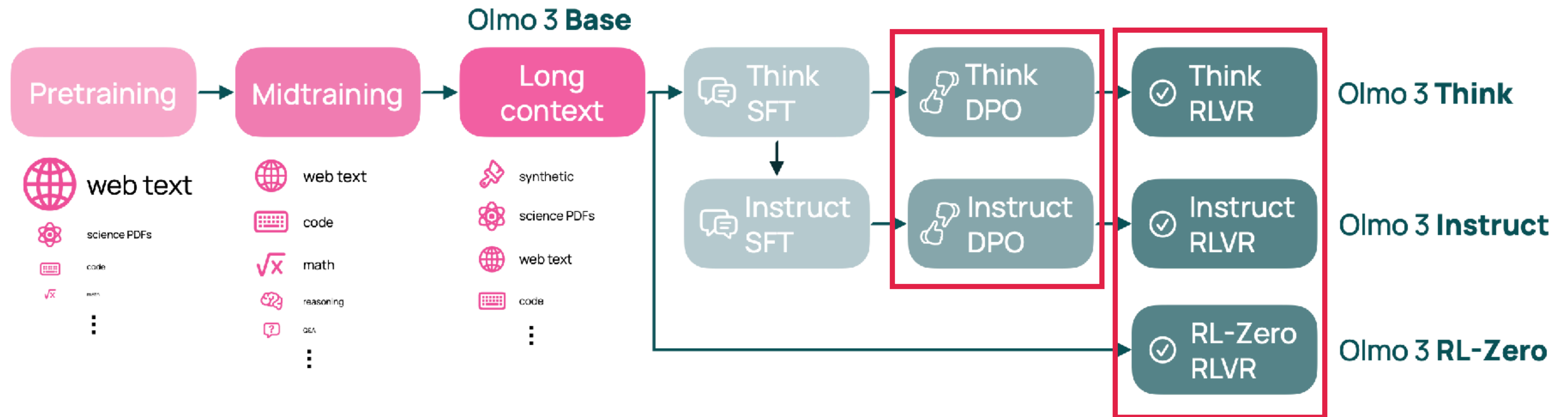
Nemotron 3 Nano
Dec 15, 2025



MiMo-V2-Flash
Dec 16, 2025

Case Study: Olmo 3

Olmo 3 uses DPO *and* RLVR:



(The think model is designed for long-form reasoning, while the instruct model is trained to give more concise answers. Mostly same objective, just different data.)

Case Study: Olmo 3

Datasets used for the RLVR stages of Olmo 3:

Category	Prompt Dataset	# Prompts Used in Think RL	# Prompts Used in Instruct RL	Reference
Precise IF	IF-RLVR	30,186	38,000	Pyatkin et al. (2025)
Math	Open-Reasoner-Zero	3,000	14,000	Hu et al. (2025)
	DAPO-Math	2,584	7,000	Yu et al. (2025)
	AceReason-Math	6,602	–	Chen et al. (2025)
	Polaris-Dataset	–	14,000	An et al. (2025)
	KlearReasoner-MathSub	3,000	9,000	Su et al. (2025c)
	OMEGA-train	15,000	20,000	Sun et al. (2025)
Coding	AceCoder	9,767	20,000	Zeng et al. (2025a)
	KlearReasoner-Code	8,040	–	Su et al. (2025c)
	Nemotron Post-training Code	2,303	–	NVIDIA AI (2025)
	SYNTHETIC-2	3,000	–	PrimeIntellect (2025)
General Chat	Tulu 3 SFT	7,129	18,955	Lambert et al. (2024)
	Wildchat-4.8M	7,129	18,761	-
	Multi-Subject RLVR	7,129	12,234	Su et al. (2025b)
Total		104,869	171,950	

Does RL teach models new things, or elicit existing abilities?

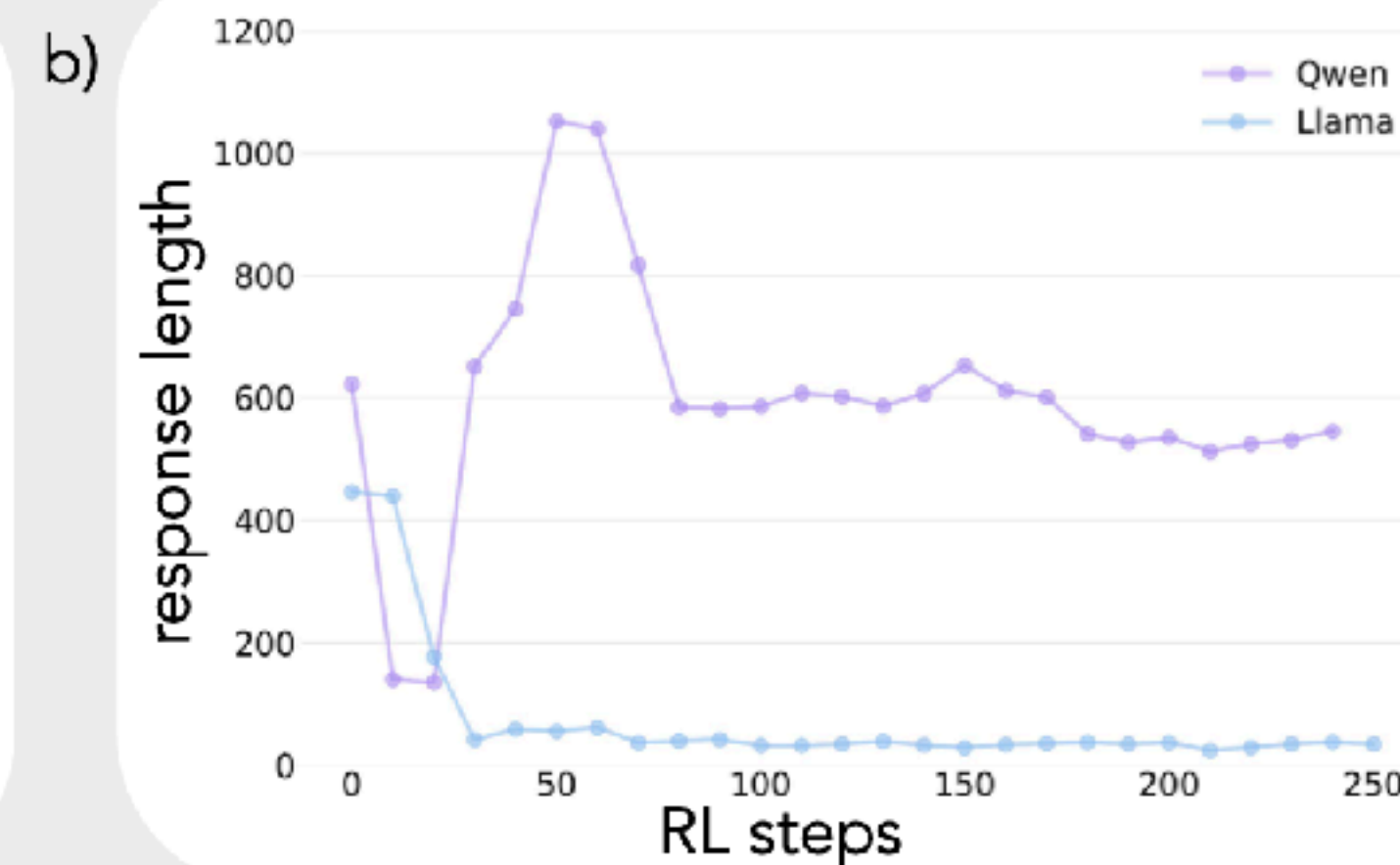
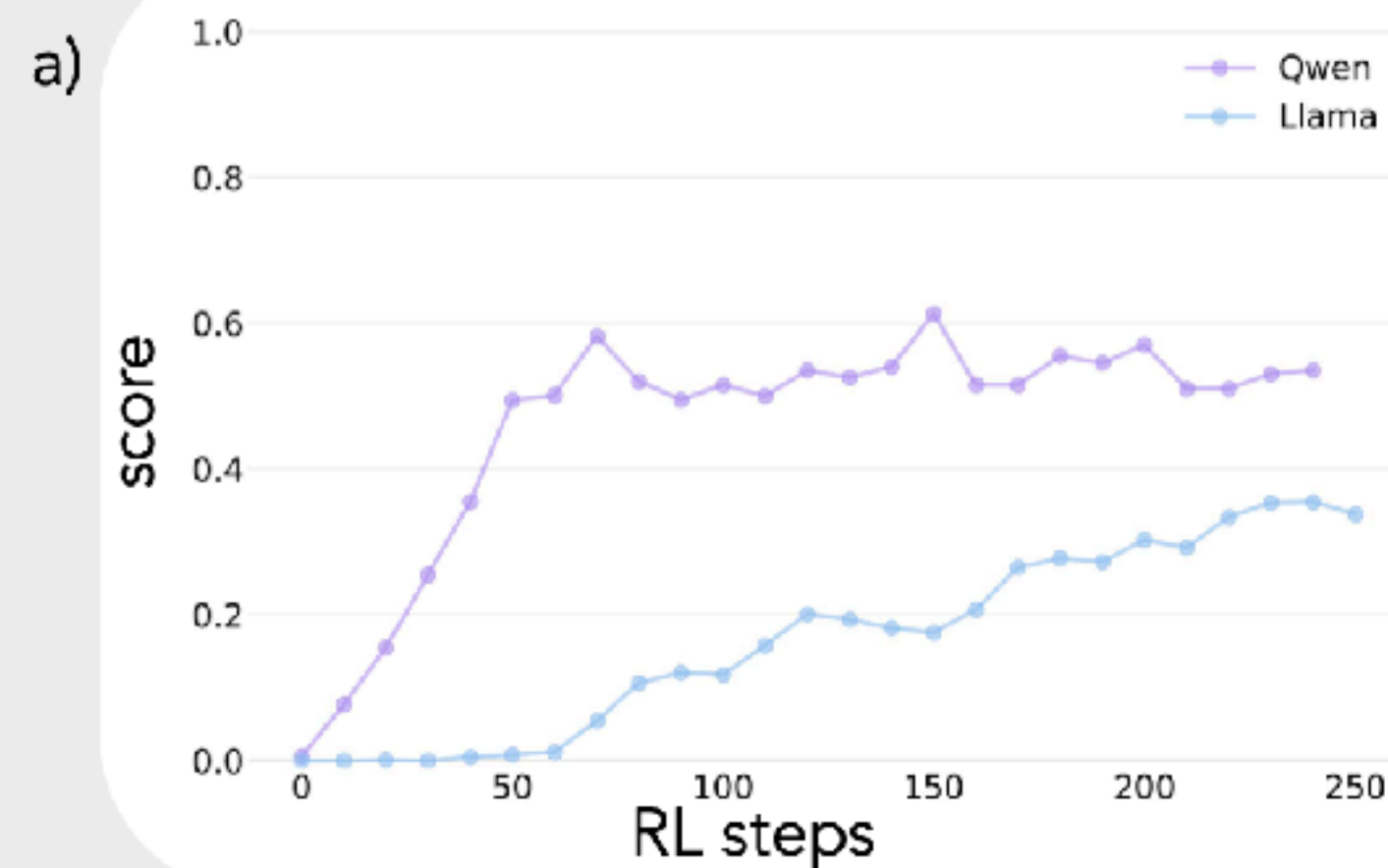
A tale of two models: Qwen 2.5 3B and Llama 3.2 3B



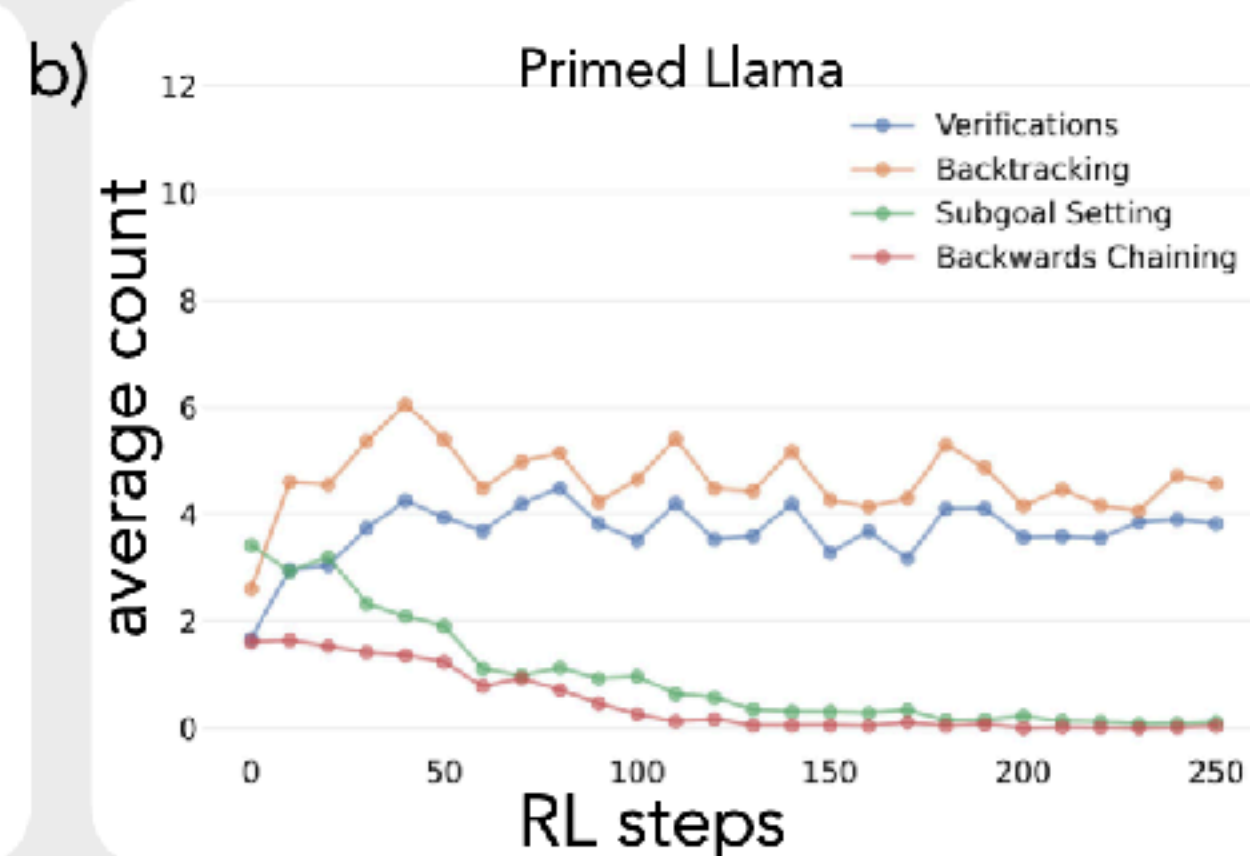
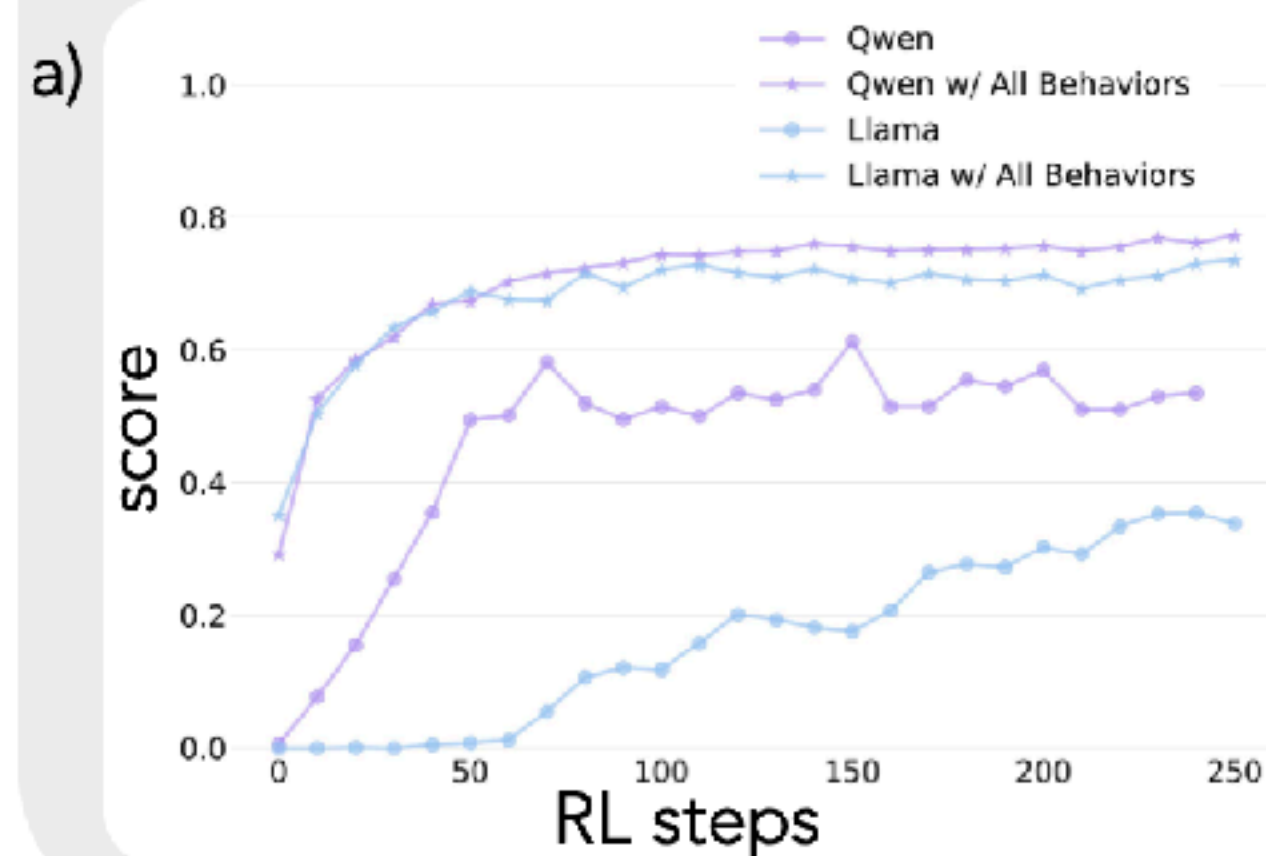
Let's start with the sum of the largest two numbers and then subtract the smallest two: $84 + 83 - 34 - 72$. This gives us $167 - 76$ which equals 91. That's not 39. Let's try another combination: $84 + 83 - 72 - 34$. This gives us $167 - 106$ which equals 61. That's still not 39. Let's try $84 + 72 - 83 - 34$. This gives us $156 - 117$ which equals 39. This is the correct equation.



84 is the difference between 108 and 34.
<answer> $(84 - 34) / 108$ </answer>



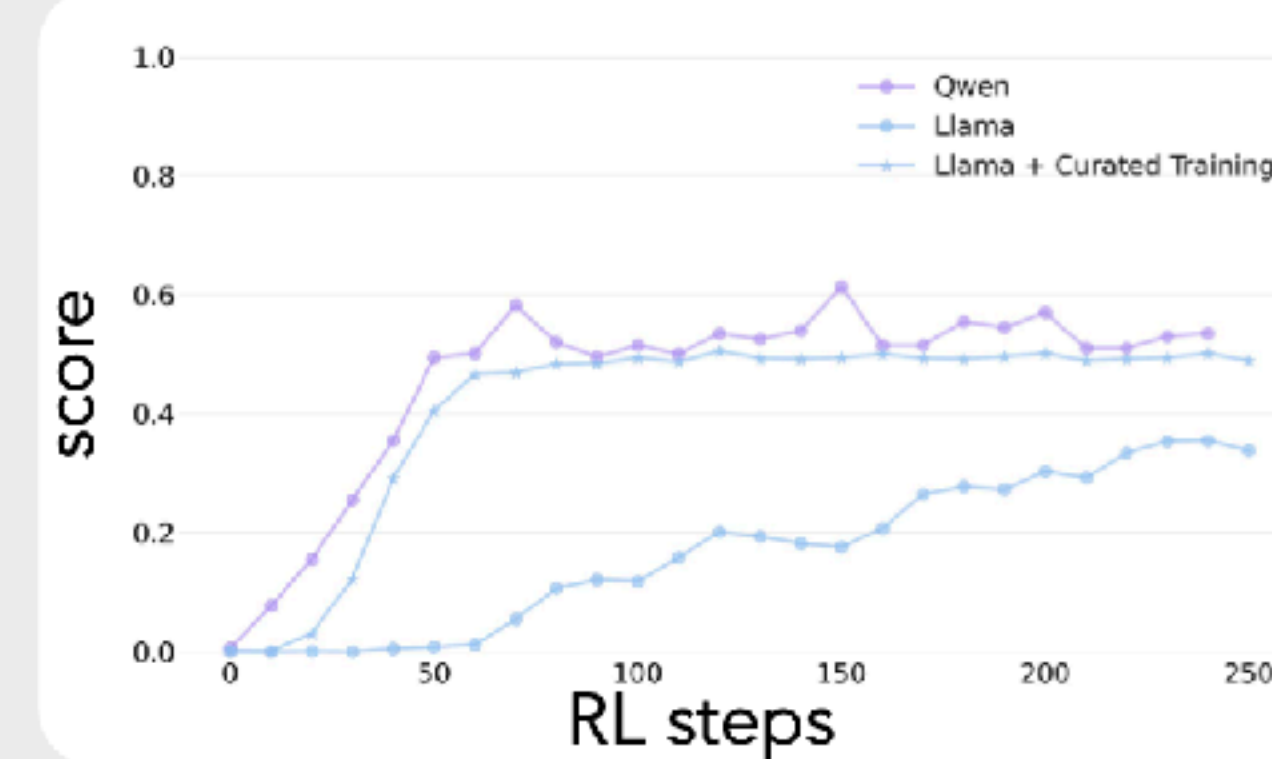
Priming with behaviors reduces performance gap



We can curate a continued pre-training set so that Llama shows similar improvements to Qwen



Filter for behaviors
Reformat as:
Query, Thought,
Answer



Dr. GRPO

- GRPO:

$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|\mathbf{o}_i|} \sum_{t=1}^{|\mathbf{o}_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right] \right\}$$

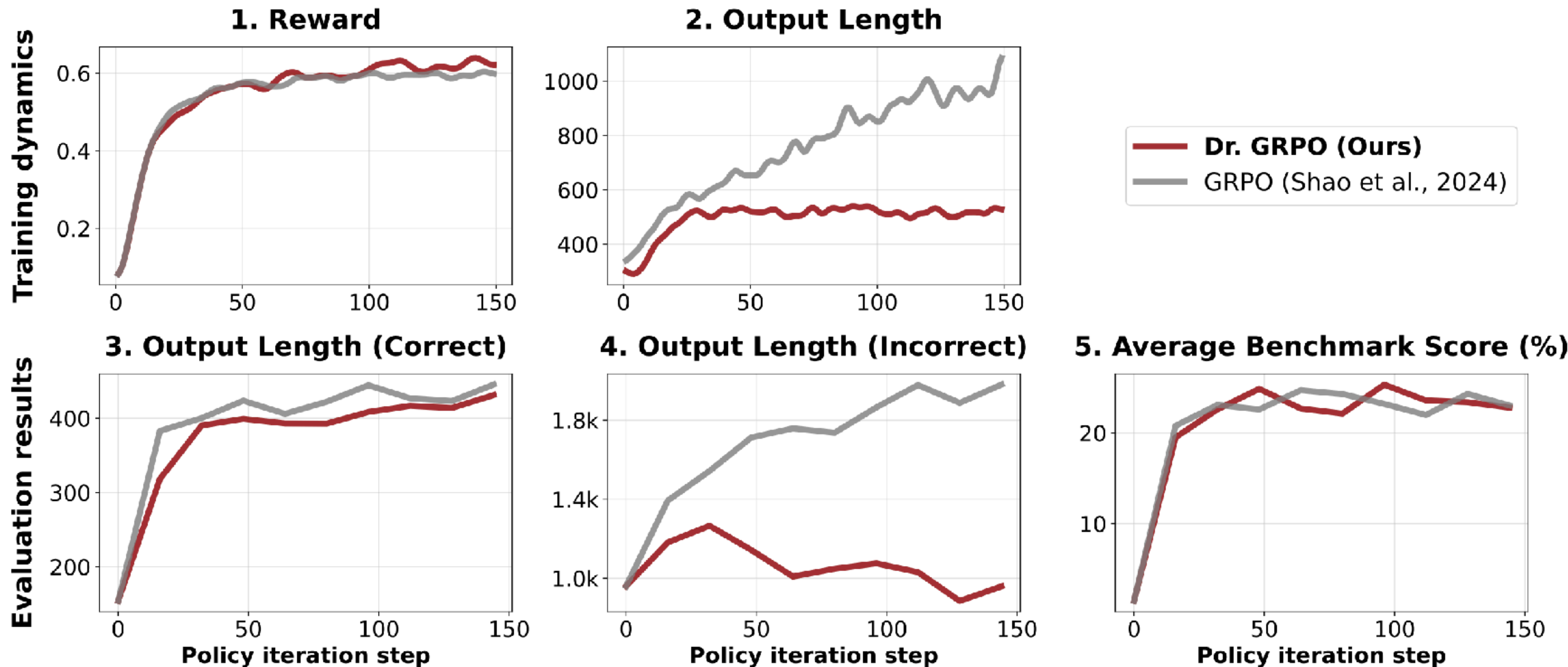
$$\text{where } \hat{A}_{i,t} = \frac{r(\mathbf{q}, \mathbf{o}_i) - \text{mean}(\{r(\mathbf{q}, \mathbf{o}_1), \dots, r(\mathbf{q}, \mathbf{o}_G)\})}{\text{std}(\{r(\mathbf{q}, \mathbf{o}_1), \dots, r(\mathbf{q}, \mathbf{o}_G)\})}$$

- GRPO **D**one **r**ight (Dr. GRPO): just remove the bias and length normalization

$$\frac{1}{G} \sum_{i=1}^G \sum_{t=1}^{|\mathbf{o}_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right] \right\}$$

$$\text{where } \hat{A}_{i,t} = r(\mathbf{q}, \mathbf{o}_i) - \text{mean}(\{r(\mathbf{q}, \mathbf{o}_1), \dots, r(\mathbf{q}, \mathbf{o}_G)\})$$

Dr. GRPO



Dr. GRPO

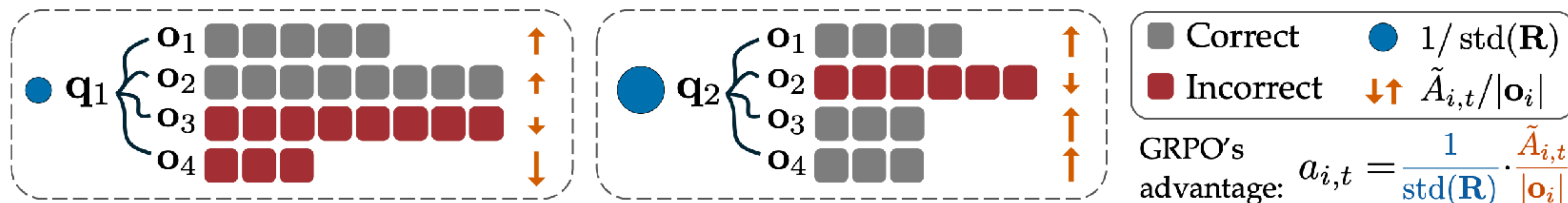


Figure 4: Illustration of the biases in GRPO. Note that the effective advantage of GRPO $a_{i,t}$ is equivalent to a reweighted version of the unbiased advantage $\tilde{A}_{i,t} = R(\mathbf{q}, \mathbf{o}_i) - \text{mean}(\mathbf{R})$. The terms $\text{std}(\mathbf{R})$ and $|\mathbf{o}_i|$ could bias the optimization by assigning different weights to different questions and responses, as denoted by the sizes of the blue circles and the lengths of the orange arrows. Upward arrows indicate positive advantages, and vice versa.

- With regular GRPO, shorter correct responses are boosted more, and longer incorrect responses are penalized less. Dr. GRPO fixes this
- Dr. GPRO also excludes the KL

Answer Parsing

- As-is, your model could generate the answer anywhere and in any format
- DeepSeek-R1-Zero uses `<answer> ... </answer>` tokens to mark the answer
- In HW2, you'll get a sense for how often answer parsing errors can drive issues

Why so many reasoning models now?

- Post-training methods are now accessible enough for widespread adoption
- Base models are good enough such that post-training can elicit actually interesting capabilities
- Verifiable rewards are unambiguous, which helps us reduce reward hacking